The University of Azad Jammu and Kashmir

Name: Zarnab Fatima

Roll NO: 2024-SE-14

Semester:4th

Course Title: Database System

Course Code: CS-22034

Submitted To: Engr. Awais Rathore

Department of Software Engineering

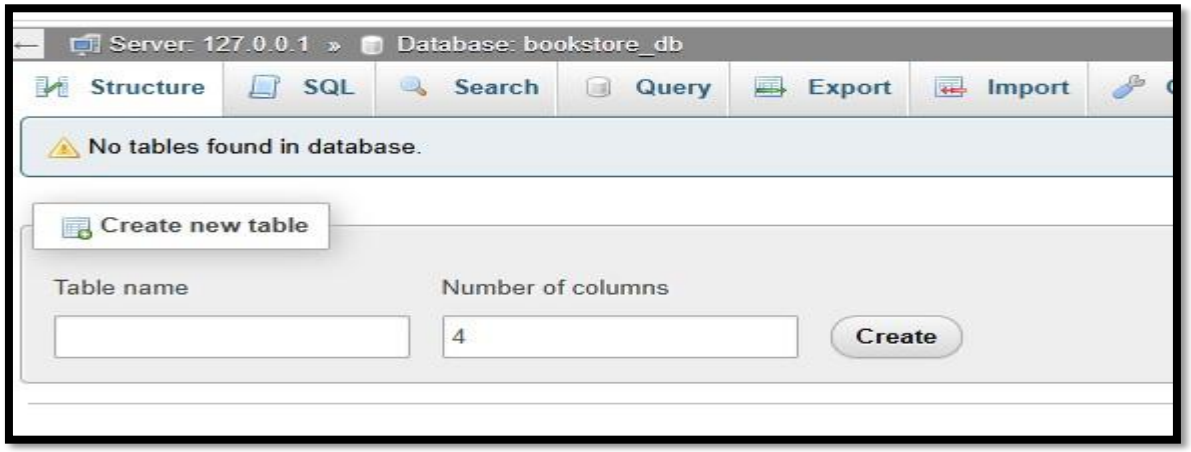
LAB 04

Task 1: Identification of Functional Dependencies and Anomalies

From the given unnormalized data, the following functional dependencies can be identified:

- **OrderID →** OrderDate, CustID
- **CustID →** CustName, CustEmail
- **BookID →** BookTitle, Publisher, UnitPrice
- (**OrderID, BookID**) → Qty

This design suffers from several anomalies. An insertion anomaly occurs because a new book or customer cannot be added without creating an order. An update anomaly exists because book and customer information is repeated across multiple rows, requiring multiple updates for a single change. A deletion anomaly occurs when deleting an order may result in loss of important customer or book information



Show query box

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0012 seconds.)

```
CREATE TABLE OrderBook_1NF ( OrderID VARCHAR(10), OrderDate DATE, CustID VARCHAR(10), CustName VARCHAR(50), CustEmail VARCHAR(50), BookID VARCHAR(10), BookTitle VARCHAR(100), Publisher VARCHAR(50), UnitPrice INT, Qty INT, PRIMARY KEY (OrderID, BookID) );
```

[Edit inline] [Edit] [Create PHP code]

Task 2: Conversion to First Normal Form (1NF)

- To convert the data into First Normal Form (1NF), all repeating groups and multivalued attributes were removed. Each row now represents a single book within an order, ensuring atomic values.
- The primary key for this table is a composite key: (OrderID, BookID)

```
SELECT * FROM `orderbook_1nf`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	OrderID	OrderDate	CustID	CustName	CustEmail	BookID	BookTitle	Publisher	UnitPrice	Qty
Edit Copy Delete	O-501	2026-04-02	C-11	Bilal	bilal@x.com	B-1	SQL Basics	Pearson	1200	1
Edit Copy Delete	O-501	2026-04-02	C-11	Bilal	bilal@x.com	B-2	Python 101	OReilly	1500	2
Edit Copy Delete	O-502	2026-04-03	C-12	Areeba	areeba@x.com	B-1	SQL Basics	Pearson	1200	3
Edit Copy Delete	O-503	2026-04-05	C-11	Bilal	bilal@x.com	B-2	Python 101	OReilly	1500	1
Edit Copy Delete	O-503	2026-04-05	C-11	Bilal	bilal@x.com	B-3	Networks	Pearson	1800	1

Task 3: Conversion to Second Normal Form (2NF)

In the 1NF table, partial dependencies were present because non-key attributes depended only on part of the composite key. For example, customer details depended only on CustID, and book details depended only on BookID.

To eliminate these partial dependencies, the table was decomposed into the following relations:

- **Customers**(CustID, CustName, CustEmail)
- **Orders**(OrderID, OrderDate, CustID)
- **Books**(BookID, BookTitle, Publisher, UnitPrice)
- **OrderDetails**(OrderID, BookID, Qty)

This ensures that all non-key attributes are fully dependent on the primary key.

Table: Customers

Table: Orders

```
SELECT * FROM Orders;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort b

Extra options

	OrderID	OrderDate	CustID
Edit Copy Delete	O-501	2026-04-02	C-11
Edit Copy Delete	O-502	2026-04-03	C-12
Edit Copy Delete	O-503	2026-04-05	C-11

Check all *With selected:* Edit Copy Delete Export

Table: Books

```
SELECT * FROM Books;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

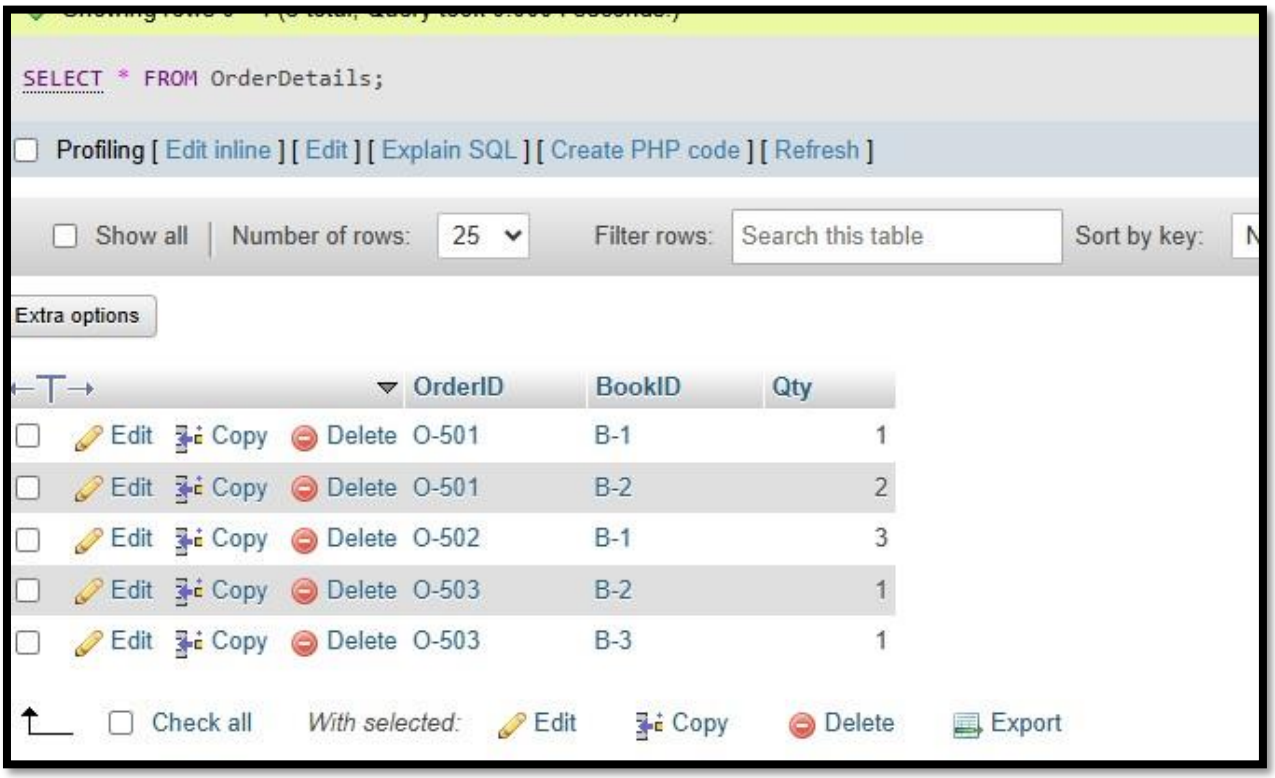
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	BookID	BookTitle	Publisher	UnitPrice
Edit Copy Delete	B-1	SQL Basics	Pearson	1200
Edit Copy Delete	B-2	Python 101	OReilly	1500
Edit Copy Delete	B-3	Networks	Pearson	1800

Check all *With selected:* Edit Copy Delete Export

Table: OrderDetails



Task 4: Conversion to Third Normal Form (3NF)

In the 2NF design, a transitive dependency still existed in the Books table, where Publisher was dependent on BookID. To remove this dependency, a separate Publishers table was created.

The final 3NF relations are:

- **Customers**(CustID, CustName, CustEmail)
- **Orders(**OrderID, OrderDate, CustID)
- **Publishers**(PublisherID, PublisherName)
- **Books**(BookID, BookTitle, PublisherID, UnitPrice)
- **OrderDetails**(OrderID, BookID, Qty)

This structure eliminates transitive dependencies and ensures data integrity.

Create: Publishers

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0015 seconds.)

```
CREATE TABLE Publishers ( PublisherID INT AUTO_INCREMENT PRIMARY KEY, PublisherName VARCHAR(50) );
```

[Edit inline] [Edit] [Create PHP code]

Drop TABLE OrderDetails: Child

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0007 seconds.)

```
DROP TABLE OrderDetails;
```

[Edit inline] [Edit] [Create PHP code]

Drop TABLE Books: Parent

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0010 seconds.)

```
DROP TABLE Books;
```

[Edit inline] [Edit] [Create PHP code]

Create: New Book

```
SELECT * FROM `books`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

Extra options

	BookID	BookTitle	PublisherID	UnitPrice
Edit Copy Delete	B-1	SQL Basics	1	1200
Edit Copy Delete	B-2	Python 101	2	1500
Edit Copy Delete	B-3	Networks	1	1800

Insert INTO Publishers & Books

2 rows inserted.
Inserted row id: 2 (Query took 0.0407 seconds.)

```
INSERT INTO Publishers (PublisherName) VALUES ('Pearson'), ('OReilly');
```

[Edit inline] [Edit] [Create PHP code]

3 rows inserted.
Inserted row id: 3 (Query took 0.0429 seconds.)

```
INSERT INTO Books VALUES ('B-1','SQL Basics',1,1200), ('B-2','Python 101',2,1500), ('B-3','Networks',1,1800);
```

[Edit inline] [Edit] [Create PHP code]

Create: OrderDetails

Show query box

MySQL returned an empty result set (i.e. zero rows). (Query took 0.0013 seconds.)

```
CREATE TABLE OrderDetails ( OrderID VARCHAR(10), BookID VARCHAR(10), Qty INT, PRIMARY KEY (OrderID, BookID), FOREIGN KEY (OrderID) REFERENCES Orders(OrderID), FOREIGN KEY (BookID) REFERENCES Books(BookID) );
```

Edit inline] [Edit] [Create PHP code]

Insert INTO OrderDetails

Show query box

5 rows inserted. (Query took 0.0538 seconds.)

```
INSERT INTO OrderDetails VALUES ('O-501','B-1',1), ('O-501','B-2',2), ('O-502','B-1',3), ('O-503','B-3',1), ('O-503','B-2',1);
```

Edit inline] [Edit] [Create PHP code]

Create: Order

```
SELECT * FROM Orders;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

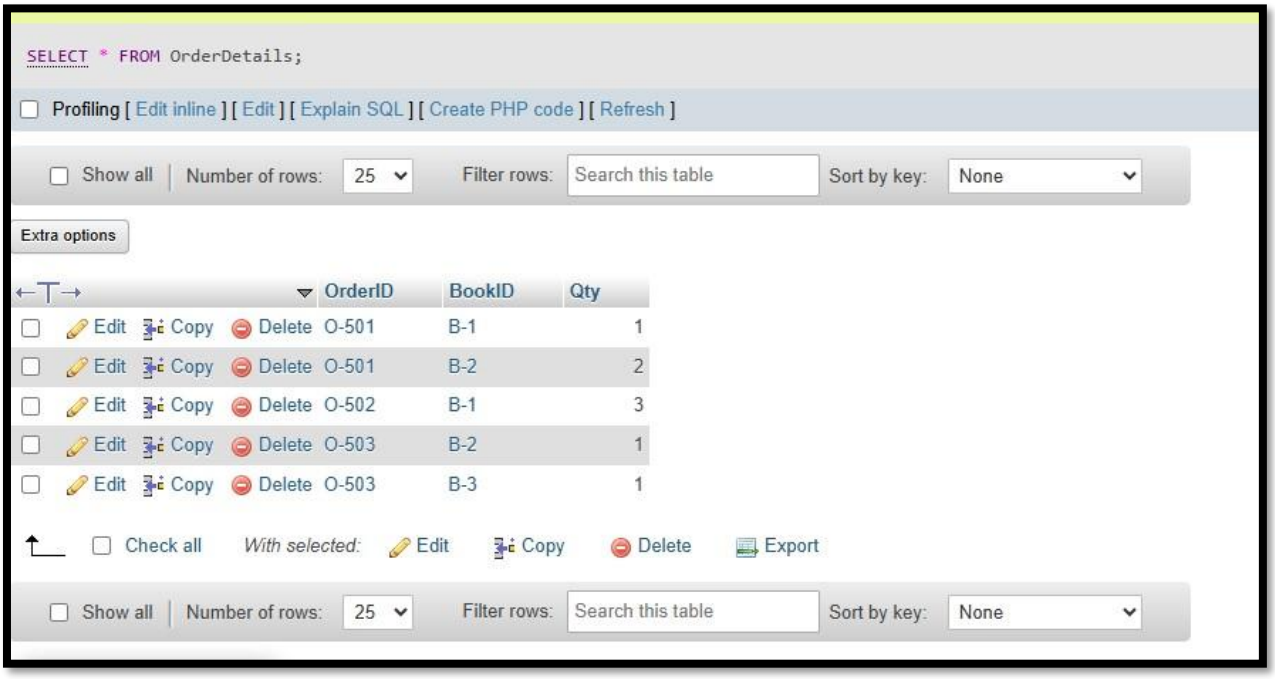
Extra options

	OrderID	OrderDate	CustID
Edit Copy Delete	O-501	2026-04-02	C-11
Edit Copy Delete	O-502	2026-04-03	C-12
Edit Copy Delete	O-503	2026-04-05	C-11

Check all *With selected:* Edit Copy Delete Export

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Table: OrderDetails



Task 5: Verification Queries

The following query joins multiple tables to reproduce the original report, showing one row per book purchased in each order.

```
SELECT
o.OrderID,o.OrderDate,c.CustName,c.CustEmail,b.BookTitle,p.PublisherName,
b.UnitPrice.od.Qty, FROM OrderDetails od JOIN
Orders o ON od.OrderID = o.OrderID
JOIN Customers c ON o.CustID = c.CustID
JOIN Books b ON od.BookID = b.BookID
JOIN Publishers p ON b.PublisherID = p.PublisherID;
```

Showing rows 0 - 4 (5 total, Query took 0.0011 seconds.)

```
SELECT o.OrderID, o.OrderDate, c.CustName, c.CustEmail, b.BookTitle, p.PublisherName, b.UnitPrice, od.Qty FROM OrderDetails od JOIN Orders o ON od.OrderID = o.OrderID JOIN Customers c ON o.CustID = c.CustID JOIN Books b ON od.BookID = b.BookID JOIN Publishers p ON b.PublisherID = p.PublisherID;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

OrderID	OrderDate	CustName	CustEmail	BookTitle	PublisherName	UnitPrice	Qty
O-501	2026-04-02	Bilal	bilal@x.com	SQL Basics	Pearson	1200	1
O-503	2026-04-05	Bilal	bilal@x.com	Networks	Pearson	1800	1
O-502	2026-04-03	Areeba	areeba@x.com	SQL Basics	Pearson	1200	3
O-501	2026-04-02	Bilal	bilal@x.com	Python 101	OReilly	1500	2
O-503	2026-04-05	Bilal	bilal@x.com	Python 101	OReilly	1500	1

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

The following query calculates the total amount spent by each customer.

```
SELECT
c.CustName,
SUM(b.UnitPrice * od.Qty) AS TotalSpend
FROM OrderDetails od
JOIN Orders o ON od.OrderID = o.OrderID
JOIN Customers c ON o.CustID = c.CustID
JOIN Books b ON od.BookID = b.BookID
GROUP BY c.CustName;
```

Showing rows 0 - 1 (2 total, Query took 0.0010 seconds.)

```
SELECT c.CustName, SUM(b.UnitPrice * od.Qty) AS TotalSpend FROM OrderDetails od JOIN Orders o ON od.OrderID = o.OrderID JOIN Customers c ON o.CustID = c.CustID JOIN Books b ON od.BookID = b.BookID GROUP BY c.CustName;
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table

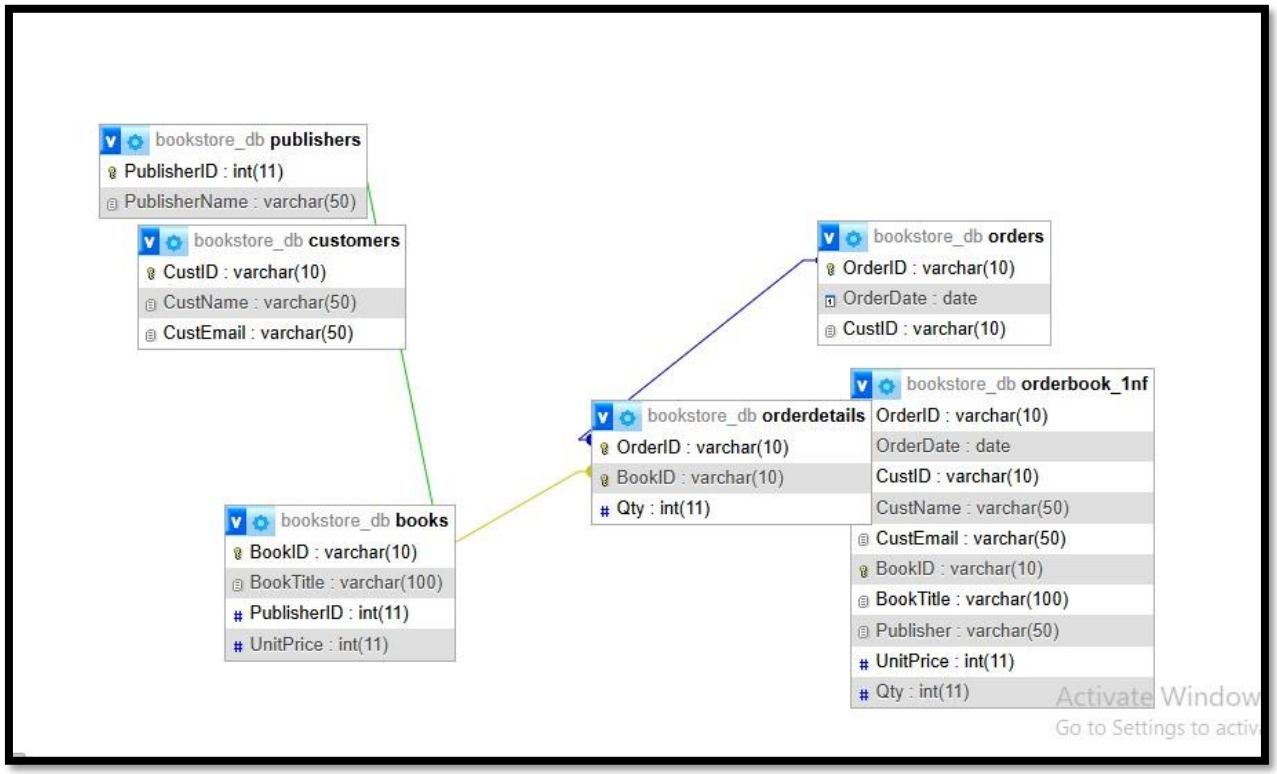
Extra options

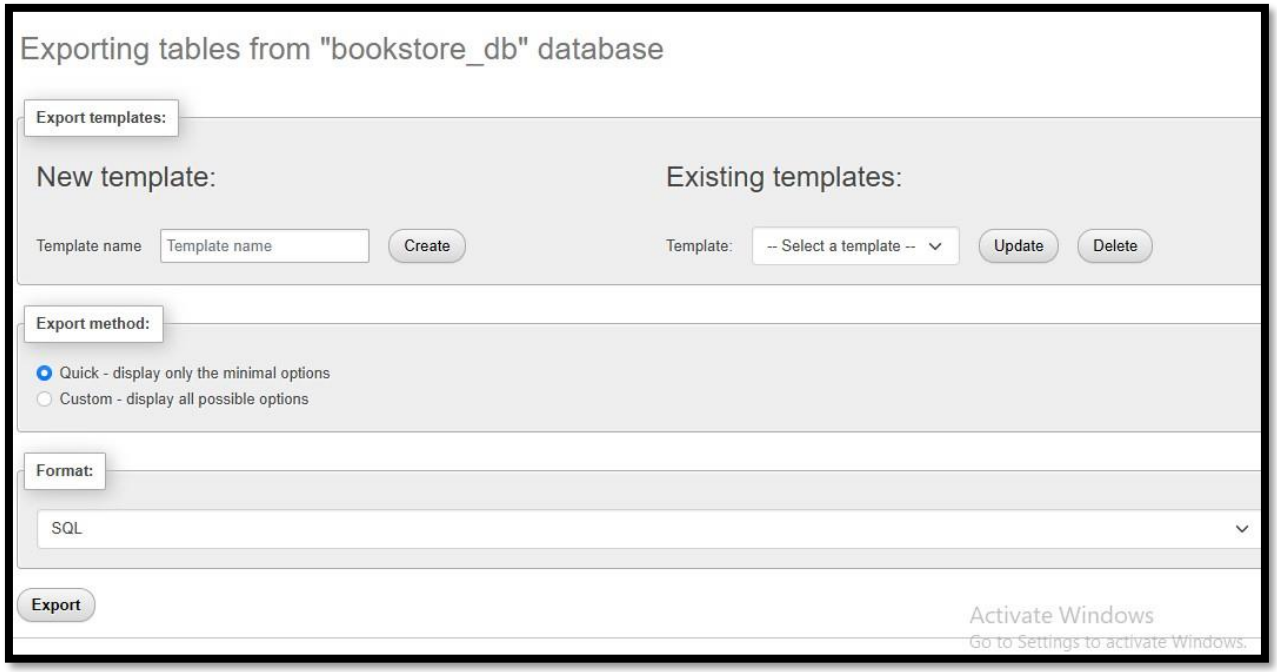
CustName	TotalSpend
Areeba	3600
Bilal	7500

Task 6: Reflection

The final 3NF schema removes redundancy by separating data into different related tables. This prevents update anomalies because changes are made in only one place. It eliminates insertion anomalies by allowing independent insertion of customers, books, and publishers. Deletion anomalies are also avoided, as deleting an order does not remove important customer or book data. Overall, normalization improves data consistency and integrity.

ER Diagram





Conclusion

- In this lab, the given unnormalized data was successfully transformed into First, Second, and Third Normal Forms. Each step of normalization removed redundancy and improved the structure of the database. Functional dependencies were identified to guide the decomposition process, ensuring that all attributes are properly related.
- The final 3NF design eliminates insertion, update, and deletion anomalies, resulting in a more reliable and efficient database system. Additionally, SQL queries were used to verify the correctness of the design and to generate meaningful results. Overall, this lab demonstrates the importance of normalization in maintaining data integrity and consistency.

*****THE END*****